Elice DevOps Infrastructure

https://github.com/jueunyeo/elice-devops-assignment

본 프로젝트는 대규모 트래픽 처리가 가능한 **고가용성(High Availability)** 인프라와 **선언적 GitOps** 체계를 구축한 DevOps 레퍼런스 모델입니다. AWS 클라우드와 On-premise(vSphere) 환경을 각각 구성하였으며, 인프라의 모든 요소는 코드(IaC)로 관리됩니다.

🏗️ 1. MSA Service Architecture

전체 서비스는 마이크로서비스 아키텍처로 설계되어 독립적인 확장과 배포를 지원합니다.

Service Component

- **API Gateway**: 모든 외부 요청의 진입점(Entrypoint)이며, 라우팅 및 통합 인증을 담당합니다.
- **Portal Web**: 사용자와 상호작용하는 프론트엔드 서비스입니다.
- **User API**: 사용자 계정, 권한 및 프로필 관리를 담당하는 핵심 비즈니스 로직 서비스입니다.
- **Media Manager**: 대용량 미디어 파일 처리 및 S3/MinIO 스토리지 연동을 전담합니다.

Traffic Flow

1. **Ingress/LoadBalancer**: 외부 트래픽을 수용하여 서비스로 전달합니다.
2. **Internal LB**: 내부 서비스 간 통신 시 로드밸런싱을 수행합니다.
3. **Persistence Layer**: 환경 특성에 따라 RDS(AWS) 또는 Postgres-HA(On-prem)로 데이터를 영속화합니다.

📊 2. Public vs Private Cloud Infrastructure Comparison

본 프로젝트는 하이브리드 클라우드 전략에 따라 각 환경에 최적화된 스택을 채택하여 운영 안정성과 제어권을 확보했습니다.

구분	Public Cloud (AWS Managed)	Prviate Cloud (On-Premise)	비고
Kubernetes	EKS (Managed)	Self-managed K8s	클라우드는 관리 효율성, 온프레미스는 커스텀 제어 중심
Database	Amazon RDS (Multi-AZ)	Postgres-HA (Patroni)	양쪽 모두 데이터 고가용성(HA) 및 자동 페일오버 확보
Storage	Amazon S3	MinIO (S3 Compatible)	동일한 S3 프로토콜을 사용하여 앱 수정 없이 호환성 유지

Secrets	**AWS Secrets Manager**	**HashiCorp Vault**	External Secrets Operator(ESO)로 시크릿 주입 방식 통일
Image Registry	**Amazon ECR**	**Harbor**	온프레미스 내 로컬 이미지 캐싱 및 보안 취약점 스캐닝 강화
Network/LB	**AWS ALB/NLB**	**MetalLB**	가상화 환경 내 소프트웨어 정의 로드 밸런싱 구현
Scaling	**Cluster Autoscaler**	**VM Provisioning**	클라우드는 노드 자동 확장, 온프레미스는 자원 고정 활용

(1) 관리 편의성 (Management Convenience)

- **AWS Managed**: 인프라 하위 계층(물리 서버, 하이퍼바이저, 네트워크 가동)의 운영 부담을 AWS가 전담합니다. 특히 EKS, RDS와 같은 Managed 서비스를 통해 OS 패치, 백업, 고가용성 구성을 자동화하여 엔지니어는 비즈니스 로직 및 애플리케이션 최적화에만 집중할 수 있는 **운영 효율성**을 제공합니다.
- **On-premise**: 서버 랙 실장부터 케이블링, vSphere 하이퍼바이저 관리, OS 커널 업데이트까지 전체 스택(Full Stack Ownership)을 내부 인력이 직접 관리해야 합니다. 하드웨어 장애 시 부품 조달 및 교체 등 **물리적 유지보수에 상당한 시간과 인력**이 소모되어 운영 부하가 상대적으로 높습니다.

(2) 민첩성 (Agility)

- **AWS Managed**: 'On-demand' 자원 할당을 통해 수 분 내에 수천 개의 리소스를 프로비저닝할 수 있습니다. 이는 신규 비즈니스의 **시장 진입 속도(Time-to-Market)를 극대화**하며, 필요 시 즉시 리소스를 삭제할 수 있어 가설 검증을 위한 실험과 실패 비용을 최소화하는 데 최적화되어 있습니다.
- **On-premise**: 신규 자원 확장을 위해 견적 요청(RFQ)부터 구매 승인, 배송 및 설치까지 **수주에서 수개월의 리드타임(Lead Time)**이 발생합니다. 급격한 트래픽 변화에 즉각 대응하기 어려우며, 인프라 확장이 비즈니스 성장 속도를 따라가지 못하는 병목 현상이 발생할 수 있습니다.

(3) 비용 효율성 (Cost-Efficiency)

- **AWS Managed**: 자본 지출(CapEx)을 **운영 비용(OpEx) 모델**로 전환합니다. 초기 대규모 투자 없이 사용한 만큼 지불(Pay-as-you-go)하며, 인프라 단가 인하를 누릴 수 있습니다. **다만**, 장기적인 고부하 워크로드의 경우 예약 인스턴스(RI)나 Savings Plans를 통한 전략적 비용 최적화가 수반되어야 합니다.
- **On-premise**: 초기 하드웨어 구매에 막대한 **자본 지출(CapEx)**이 발생하지만, 장비 감가상각 기간(3~5년) 동안 일정한 고부하 워크로드를 처리할 경우 단위 자원당 비용(Unit Cost)은 클라우드보다 저렴할 수 있습니다. 단, 전력, 냉각, 공간 및 유휴 자원 유지비용을 포함한 **전체 소유 비용(TCO)** 관점에서의 철저한 분석이 필요합니다.

(4) 보안 측면 (Security)

- **AWS Managed**: 전 세계 수십 개의 보안 규제 인증(ISO 27001, SOC, HIPAA 등)을 통과한 **글로벌 수준의 물리 보안**과 자동화된 보안 도구(IAM, GuardDuty, WAF)를 제공합니다. 'Security by Design'을 통해 인프라 전반의 보안 가시성을 확보하기 용이하나, 설정 오류(Misconfiguration)에 대한 책임은 사용자에게 있습니다.
- **On-premise**: 데이터에 대한 **물리적 제어권(Data Sovereignty)**을 완벽히 보유합니다. 폐쇄망 구성을 통해 민감한 내부 기밀이나 법규상 클라우드 저장이 불가한 데이터를 관리하기에 유리합니다. 그러나 침입 탐지 시스템(IDS/IPS), 암호화 체계 등을 수동으로 구축/운영해야 하므로 전문 보안 인력 부재 시 보안 허점이 발생할 위험이 있습니다.

📂 3. Directory Structure

인프라(IaC)와 애플리케이션(K8s)의 책임을 엄격히 분리하여 관리 효율성을 높였습니다.

```
elice-devops-assignment/
├── .github/workflows/          # [CI] 자동화 파이프라인
├── aws/
│   ├── terraform/             # [IaC] AWS 인프라 정의 (VPC, EKS, RDS, S3)
│   │   ├── bootstrap/         # Backend(S3/DynamoDB) 초기 구축용
│   │   ├── environments/      # dev, stage, prod 환경별 설정
│   │   └── modules/           # 재사용 가능한 인프라 모듈 (vpc, eks, storage, database)
│   └── kubernetes/            # [GitOps] AWS EKS 전용 매니페스트
│       ├── argocd/            # ArgoCD Application 정의 (main-app.yaml)
│       ├── external-secrets/  # External Secrets Operator 설정
│       └── services/          # 각 MSA 서비스별 Helm Values
├── on-prem/
│   ├── terraform/             # [IaC] vSphere 환경 인프라 정의
│   └── kubernetes/            # [GitOps] On-prem K8s 전용 매니페스트
├── shared/
│   └── kubernetes/charts/     # [Standard] 전 서비스 공통 Helm Chart (elice-common)
└── README.md
```

🛠️ 4. Infrastructure as Code (Terraform)

인프라의 정합성(Consistency)과 재현성(Reproducibility)을 위해 모든 AWS 및 vSphere 리소스는 Terraform으로 관리됩니다.

High Availability (HA) & Scaling

- **Multi-AZ Architecture**: EKS 워커 노드 및 RDS 인스턴스를 다중 가용 영역(Multi-AZ)에 분산 배치하여 단일 데이터 센터 장애 시에도 서비스 연속성을 보장합니다.
- **Auto-scaling Strategy**:
  - **Cluster Autoscaler**: 트래픽 증가에 따른 물리 노드(EC2) 자동 증설.
  - **HPA (Horizontal Pod Autoscaler)**: CPU/Memory 메트릭 기반의 Pod 단위 자동 확장.
- **Postgres-HA**: On-premise 환경에서도 데이터 가용성을 위해 Patroni 기반의 고가용성 클러스터를 구성했습니다.

Environment Isolation

`dev`, `stage`, `prod` 환경을 디렉토리 수준에서 완전히 물리적으로 격리하였으며, 각 환경별로 독립적인 `tfvars`와 Backend 설정을 가집니다.

Multi-Account Architecture (AWS Organizations)

AWS 환경은 `dev`, `stage`, `prod`를 각각 별도 AWS Account로 분리하고, Management Account를 컨트롤 플레인으로 사용하는 구조로 운영합니다.

- **Account Isolation**: 환경 단위 계정 분리를 통해 장애 전파 범위와 권한 경계를 최소화합니다.

- **Provider Assume Role**: 각 환경 Terraform은 `OrganizationAccountAccessRole`을 Assume 하여 대상 계정에 배포합니다.
- **Environment Account ID Parameterization**: `aws_account_id` 변수를 통해 환경별 대상 Account를 선언적으로 주입합니다.

State Management & Locking

- **Backend Strategy**: S3를 원격 백엔드로 사용하여 팀 협업 시 State 공유를 지원합니다.
- **State Locking**: **DynamoDB Table**을 통한 Locking 메커니즘을 적용하여, 다수의 엔지니어가 동시에 실행할 때 발생할 수 있는 Race Condition과 State 오염을 원천 차단합니다.

🔄 5. CI/CD & GitOps Workflow

본 레포지토리는 **인프라 관리 및 GitOps 전용 레포지토리**입니다.
파이프라인을 인프라와 서비스로 분리해 안정성과 배포 민첩성을 동시에 확보합니다.

A. Infrastructure Pipeline (Atlantis + Terraform)

```
[Infra PR: aws/terraform | on-prem/terraform]
            |
            v
       [Atlantis Plan]
            |
   (PR Approved Required)
            |
            v
      [Atlantis Apply]
            |
            v
[AWS/On-prem Infra Provisioning]
```

- **실행 주체**: Atlantis (EKS 내 Pod)
- **대상 범위**: `aws/terraform/`, **`on-prem/terraform/`**
- **승인 게이트**: `apply_requirements: [approved]`
- **보안 모델**: Atlantis IRSA Role -> 각 계정 Assume Role (`OrganizationAccountAccessRole`)

B. Service Pipeline (External CI + ArgoCD)

```
[App Repo CI: Build/Test/Image Push]
            |
            v
[External GHA updates values-{env}.yaml in this repo]
            |
            v
   [ArgoCD App-of-Apps detects Git change]
            |
```

```
            v
[Auto Sync (prune + selfHeal) to clusters]
```

- **실행 주체**
  - 이미지 빌드/테스트: 외부 애플리케이션 레포 CI
  - 배포 타겟 업데이트: 이 레포의 `values-{env}.yaml`
  - 배포 적용: ArgoCD (클러스터 내부)
- **배포 타겟 경로**
  - `aws/kubernetes/services/<service>/values-{env}.yaml`
  - `on-prem/kubernetes/services/<service>/values-{env}.yaml`
- **ArgoCD 동작**
  - Root App (`main-app.yaml`)이 child app 및 values 변경을 지속 감시
  - `automated: { prune: true, selfHeal: true }` 기반 자동 동기화

Multi-repo Deployment Guide

개발자는 이 레포지토리에 직접 커밋하지 않고, 외부 애플리케이션 레포 CI에서 `repository_dispatch` API로만 배포 신호를 전달합니다.

- **Dispatch 수신 워크플로우**: `.github/workflows/service-deploy-handler.yml`
- **핵심 검증**:
  - 허용된 서비스(`api-gateway` , `portal-web` , `user-api` , `media-manager`)만 처리
  - 허용된 환경(`dev` , `stage` , `prod`)만 처리
  - 대상 파일 존재 여부 확인 후 `.image.tag` 업데이트
- **커밋 메시지 규칙**:
  - `[Deploy] <service> to <env> with tag <tag> [skip ci]`

Helm Chart Standardization

- **elice-common**: 모든 마이크로서비스가 공통으로 사용하는 Standard Helm Chart입니다.

🔐 6. Security

보안은 설계 단계에서부터 고려된 **Security by Design** 원칙을 따릅니다.

1) EKS Control Plane Endpoint Isolation

- EKS API Endpoint는 `endpoint_public_access=true` 를 유지하되, `public_access_cidrs` 를 통해 허용된 CIDR에서만 접근 가능하도록 제한했습니다.
- 동시에 `endpoint_private_access=true` 를 적용해 내부 노드/워크로드와의 통신 효율 및 보안성을 강화했습니다.
- 환경별로 `eks_public_access_cidrs` 변수를 통해 접근 대역을 분리 관리합니다.

2) Data Encryption & Deletion Safety

- **RDS**
  - `storage_encrypted=true` 적용

    - AWS 관리형 기본키 대신 서비스 전용 `aws_kms_key` 생성 후 연결
    - `deletion_protection=true` 로 실수 삭제 방지
- **S3**
    - SSE를 KMS 기반(`aws:kms`)으로 전환
    - 버킷별 전용 `aws_kms_key` 와 alias를 생성해 연결
    - `bucket_key_enabled=true` 로 KMS 비용/성능 효율 개선

3) Zero Trust Network Policy

- `aws/kubernetes/services/` * 및 `on-prem/kubernetes/services/` *에 서비스별 `network-policy.yaml` 을 추가했습니다.
- 기본 정책은 **Default Deny All**이며, 필요한 통신만 Label 기반으로 허용합니다.
- 핵심 허용 경로:
    - Ingress Controller -> `api-gateway`
    - `api-gateway` -> `user-api`
    - `user-api` -> `postgres` (TCP/5432)
    - DNS 질의(UDP/TCP 53)

4) Pod Runtime Security (PSS Restricted 방향)

- 공통 Helm 차트(`elice-common`)의 Deployment 템플릿 기본 보안 컨텍스트를 강화했습니다.
    - `runAsNonRoot: true`
    - `allowPrivilegeEscalation: false`
    - `readOnlyRootFilesystem: true`
    - `capabilities.drop: [ALL]`
- 이를 통해 기본 런타임 공격 표면(권한 상승, 쓰기 가능한 루트FS)을 축소합니다.

5) Image Security & Secret Hygiene

- ECR Repository 생성 시 `scan_on_push=true` 를 활성화해 이미지 푸시 단계에서 취약점 스캔을 수행합니다.
- Harbor values 파일에 평문 비밀번호를 두지 않도록 정책을 명시하고, External Secrets 기반 주입을 가이드했습니다.

🚀 7. Getting Started

실제 운영 환경 구축을 위한 상세 연동 절차입니다. 본 프로젝트는 로컬 실행을 지양하며 CI/CD 환경에서의 구동을 원칙으로 합니다.

Step 1: Infrastructure Bootstrap (Backend Setup)

1. **Terraform Backend 초기화 (AWS Management Account)**

- 경로: `aws/terraform/bootstrap/state-backend/`
- 동작: 관리자 권한으로 위 경로의 코드를 1회 수동 배포하여 리소스를 생성합니다.
- 목적: 중앙 S3 State Bucket + DynamoDB Lock Table 생성

2. **Atlantis 서버 배포 (AWS Management Account / EKS)**

- 경로: `aws/terraform/bootstrap/atlantis/`
- 동작: 관리자 권한으로 위 경로의 코드를 1회 수동 배포하여 리소스를 생성합니다.
- 구성:
  - Helm 기반 Atlantis 배포
  - IRSA Role 생성 및 Atlantis ServiceAccount 연결
  - 하위 계정(`dev` , `stage` , `prod`) `OrganizationAccountAccessRole` Assume 정책 부여
  - AWS Load Balancer Controller 기반 ALB Ingress (HTTPS 전용)
  - AWS Secrets Manager에서 `github_token` , `webhook_secret` 주입

3. **Harbor 서버 배포 (On-prem Kubernetes)**

- 경로: `on-prem/terraform/bootstrap/harbor/`
- 동작: 관리자 권한으로 위 경로의 코드를 1회 수동 배포하여 리소스를 생성합니다.
- 구성:
  - Helm 기반 Harbor 배포
  - `on-prem/terraform/environments/{env}/addons/values-harbor.yaml` 을 `templatefile` 로 로드
  - 기존 StorageClass를 사용하는 PVC 설정 주입
  - NGINX Ingress 또는 MetalLB/NodePort 모드 선택 가능

Step 2: External Credential Integration

각 파이프라인 실행 주체별로 권한을 분리 연동합니다.

- **Atlantis (Infra)**:
  - EKS ServiceAccount + IRSA Role 연동
  - 하위 계정 `OrganizationAccountAccessRole` Assume 권한
- **External App CI (Service)**:
  - 이 레포지토리 push 권한이 있는 GitHub Token/App
  - 변경 대상은 `values-{env}.yaml` 파일의 `image.tag` 필드로 제한

Step 3: Provisioning via Pull Request (CI)

모든 인프라 수정은 Atlantis 기반 PR 워크플로우로 검증/반영됩니다.

1. 브랜치에서 인프라 코드를 수정 후 PR을 생성합니다.
2. Atlantis가 변경된 Terraform 프로젝트 기준으로 `plan` 을 수행합니다.
3. PR 승인 후에만 `atlantis apply` 가 허용됩니다(`apply_requirements: [approved]`).

Step 4: GitOps Deployment (ArgoCD)

인프라(EKS)가 준비되면 ArgoCD를 통해 애플리케이션을 배포합니다.

1. `aws/kubernetes/argocd/main-app.yaml` 을 클러스터에 적용합니다.
2. 이는 **App-of-Apps 패턴**에 따라 `apps/` 폴더 내의 각 서비스 선언을 읽어와 전체 MSA를 구성합니다.
3. **동작 원리**: ArgoCD는 Git의 `values.yaml` 변경을 감지하고, 실제 클러스터 상태와 다를 경우 자동으로 **Self-healing**을 수행합니다.